Hamiltonian Monte Carlo with Stan and Ulam

EES 4891-06/5891-01

Bayesian Statistical Methods

Jonathan Magnolia Gilligan

Class #12: Monday February 23, 2026

Changes from the Textbook

Changes from the Textbook

- Since the textbook was published (Second Edition), there have been a few small changes to the `rethinking` package:
  1. By default, it uses the `cmdstanr` package instead of `rstan`
     - It is more straightforward and reliable to install `cmdstanr` than `rstan`
     - To extract the fitted model from an `ulam` object `mdl`, use `mdl@cstanfit` instead of `mdl@stanfit` (see p. 290).
  2. The `precis()` function operating on a model fitted by `ulam()` are slightly different:
     - `ess_bulk` (Effective sample size) instead of `n_eff`
     - `rhat` instead of `Rhat4` for the metric of how well the model has converged.
- Apart from this, you should be able to reproduce the examples given in the book.

Hamiltonian Monte Carlo Sampling

Prepare data

- `rugged` data set:
  - Each row is a different country
  - `rugged`: Topographic roughness of the terrain
  - `rpcgdp_2000`: Per-capita GDP in 2000
- Use the logarithm of GDP.
- quasi-standardize the variables.
- Model

$$
\begin{aligned}
G &\sim \text{Normal}(\mu, \sigma) \\
\mu &= \alpha_{\text{CID}} + \beta_{\text{CID}} R \\
\alpha_{\text{CID}} &\sim \text{Normal}(1, 0.1) \\
\beta_{\text{CID}} &\sim \text{Normal}(0, 0.3) \\
\sigma &\sim \text{Exponential}(1)
\end{aligned}
$$

```
library(tidyverse)
library(rethinking)

data(rugged)
d <- rugged

dd <- d %>% filter(complete.cases(rgdppc_2000)) %>%
  mutate(
    log_gdp = log(rgdppc_2000),
    log_gdp_std = log_gdp / mean(log_gdp),
    rugged_std = rugged / max(rugged),
    cid = ifelse(cont_africa == 1, 1, 2)
  )
```

```
lst_rug <- alist(
  log_gdp_std ~ dnorm(mu, sigma),
  mu <- a[cid] + b[cid] * (rugged_std - 0.215),
  a[cid] ~ dnorm(1, 0.1),
  b[cid] ~ dnorm(0, 0.3),
  sigma ~ dexp(1)
)
```

`quap` vs. `ulam`

- `quap` calculates the posterior using a *quadratic approximation*
  - Focus on finding the peak of the posterior (maximum *a-posteriori*) for each parameter.
    - Near the peak, the posterior looks like a quadratic function (parabola).
  - Approximate the width
  - Because of this quadratic approximation, the resulting posterior is always a Gaussian (Normal) distribution.
  - This approximation works well when the posterior is roughly Gaussian (Normal),
    - Not so well when it's not.

- `ulam` calculates the posterior using a *Hamiltonian Monte Carlo* sampler.
  - Generates samples from the posterior.
  - Doesn't directly calculate the *maximum a-posteriori*.
- Use statistical functions to analyze the posterior (mean, median, standard deviation, etc.)
- Runs slower than `quap` but works with a greater variety of prior and likelihood functions.

ulam and stan

```
lst_rug <- alist(
  log_gdp_std ~ dnorm(mu, sigma),
  mu <- a[cid] + b[cid] * (rugged_std - 0.215),
  a[cid] ~ dnorm(1, 0.1),
  b[cid] ~ dnorm(0, 0.3),
  sigma ~ dexp(1)
)
```

```
dat_slim <- list(log_gdp_std = dd$log_gdp_std,
                 rugged_std = dd$rugged_std,
                 cid = as.integer(dd$cid))
mdl_rug_ulam <- ulam(lst_rug, data = dat_slim,
                     chains = 4, cores = 4)
```

```
## Running MCMC with 4 parallel chains, with 1 thread(s) per
chain...
##
## Chain 1 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 1 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 1 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 1 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 1 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 1 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 1 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 1 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 1 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 1 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 1 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 1 Iteration: 1000 / 1000 [100%]  (Sampling)
```

```
## Chain 2 Iteration:   1 / 1000 [  0%]  (Warmup)
```

Ulam creates this Stan model:

```
cat(mdl_rug_ulam@model)
```

```
## data{
##     vector[170] log_gdp_std;
##     vector[170] rugged_std;
##    array[170] int cid;
## }
## parameters{
##     vector[2] a;
##     vector[2] b;
##     real<lower=0> sigma;
## }
## model{
##     vector[170] mu;
##    sigma ~ exponential( 1 );
##    b ~ normal( 0 , 0.3 );
##    a ~ normal( 1 , 0.1 );
##    for ( i in 1:170 ) {
##        mu[i] = a[cid[i]] + b[cid[i]] * (rugged_std[i] -
0.215);
##    }
##    log_gdp_std ~ normal( mu , sigma );
## }
```

Running `ulam`

- Model specification is pretty much the same as for `quap`.
- Data must **only** have the observed variables in the model
  - Do transforms (`log()`, `standardize()`, ...) before calling `ulam()`.
- When you call `ulam()` it:
  1. Generates `stan` code for the model.
  2. Calls `stan` to translate the `stan` model into C++.
  3. Compiles the C++ model.
  4. Runs the model with the data you provided.
     - Default: 1000 iterations per chain
     - Half are *warm-up* and half are samples from posterior
       - *warm-up* is where the model tunes itself
  5. Returns the results.

```
lst_rug <- alist(
  log_gdp_std ~ dnorm(mu, sigma),
  mu <- a[cid] + b[cid] * (rugged_std -
        0.215),
  a[cid] ~ dnorm(1, 0.1),
  b[cid] ~ dnorm(0, 0.3),
  sigma ~ dexp(1)
)
```

```
mdl_rug_quap <- quap(lst_rug, data =
        dat_slim)
```

```
dat_slim <- list(log_gdp_std =
        dd$log_gdp_std,
                 rugged_std = dd$rugged_std,
                 cid = as.integer(dd$cid))
mdl_rug_ulam <- ulam(lst_rug, data =
        dat_slim,
                     chains = 4, cores = 4)
```

Running `ulam`

- `chains`: A *chain* is a sequence of samples.
  - *Markov chain*: Each sample depends on the previous sample.
  - Different chains start from different starting points.
  - *Mixing*: After a certain number of samples, chains that start at different starting points should become indistinguishable.
  - You should sample multiple chains so you can check whether the chains are well-mixed.
- `cores`: How many processor cores to use:
  - When `cores = 1`, the chains execute sequentially
  - When `cores = chains`, all chains execute in parallel (faster).
  - There's no extra benefit in having more `cores` than `chains`.

```
lst_rug <- alist(
  log_gdp_std ~ dnorm(mu, sigma),
  mu <- a[cid] + b[cid] * (rugged_std -
        0.215),
  a[cid] ~ dnorm(1, 0.1),
  b[cid] ~ dnorm(0, 0.3),
  sigma ~ dexp(1)
)
```

```
dat_slim <- list(log_gdp_std =
        dd$log_gdp_std,
                 rugged_std = dd$rugged_std,
                 cid = as.integer(dd$cid))
mdl_rug_ulam <- ulam(lst_rug, data =
        dat_slim,
                     chains = 4, cores = 4)
```

Interpreting Model Results

Model Output

```
lst_rug <- alist(
  log_gdp_std ~ dnorm(mu, sigma),
  mu <- a[cid] + b[cid] * (rugged_std - 0.215),
  a[cid] ~ dnorm(1, 0.1),
  b[cid] ~ dnorm(0, 0.3),
  sigma ~ dexp(1)
)
```

```
dat_slim <- list(log_gdp_std = dd$log_gdp_std,
                 rugged_std = dd$rugged_std,
                 cid = as.integer(dd$cid))
mdl_rug_ulam <- ulam(lst_rug, data = dat_slim,
                     chains = 4, cores = 4)
```

- For each parameter, `precis` shows:
  - Statistical properties: mean, standard deviation, 5.5% and 94.5% quantiles,
  - `ess_bulk`: The effective sample size (smaller than the actual number of samples because of autocorrelation)
  - `rhat`: The Gelman-Rubin convergence diagnostic:
    - variance between chains / variance within chain
    - 1 when chains have converged completely
    - ≤ 1.1 indicates good convergence (well-mixed)

```
show(mdl_rug_ulam)
```

```
## Hamiltonian Monte Carlo approximation
## 2000 samples from 4 chains
##
## Sampling durations (seconds):
##   chain_id warmup sampling total
## 1        1   0.05     0.03  0.09
## 2        2   0.05     0.04  0.09
## 3        3   0.05     0.05  0.10
## 4        4   0.05     0.04  0.08
##
## Formula:
## log_gdp_std ~ dnorm(mu, sigma)
## mu <- a[cid] + b[cid] * (rugged_std - 0.215)
## a[cid] ~ dnorm(1, 0.1)
## b[cid] ~ dnorm(0, 0.3)
## sigma ~ dexp(1)
```

```
precis_show(precis(mdl_rug_ulam, depth = 2, digits = 2))
```

```
##        mean   sd  5.5% 94.5% rhat ess_bulk
## a[1]   0.89 0.02  0.86  0.91    1  2901.28
## a[2]   1.05 0.01  1.03  1.07    1  3186.68
## b[1]   0.13 0.08  0.01  0.26    1  2752.39
## b[2]  -0.14 0.05 -0.23 -0.05    1  2340.66
## sigma  0.11 0.01  0.10  0.12    1  2788.01
```

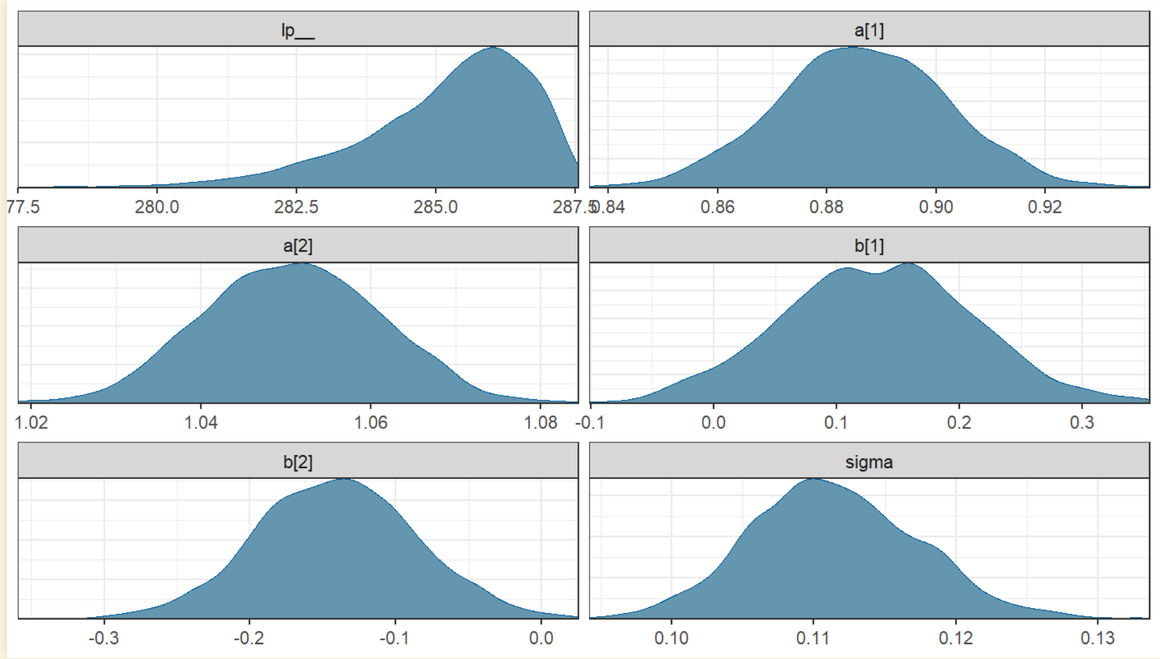
Plotting Model Results

```
library(bayesplot)

mdl_fit <- mdl_rug_ulam@cstanfit
mdl_draws <- mdl_fit$draws()
```

```
mcmc_dens(mdl_draws)
```

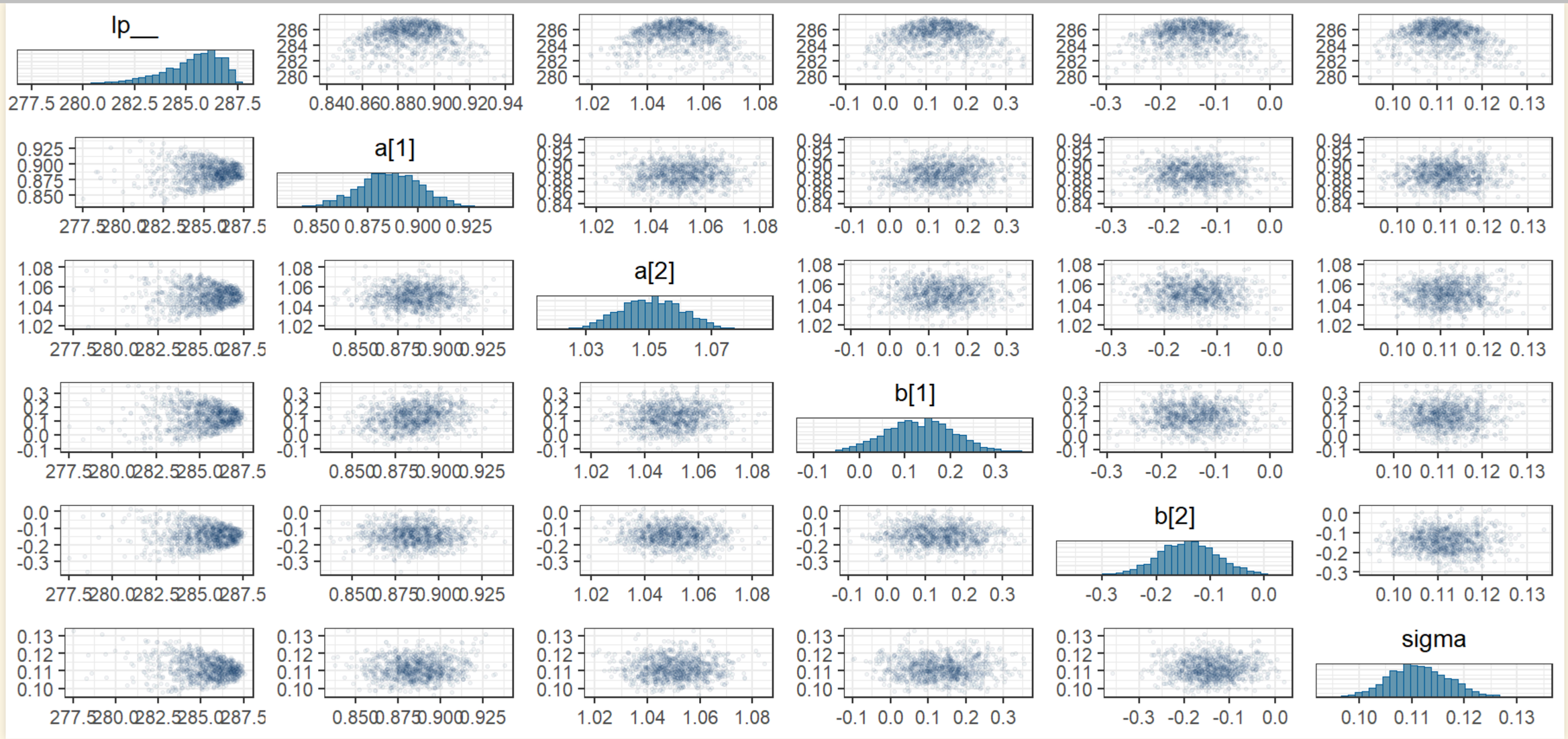
- The graphs show the probability density of each parameter.
- `lp__` is the log of the posterior probability density.
  - This graph shows how the MCMC samples are mostly drawn from higher probability regions of the posterior.



Model Diagnostics

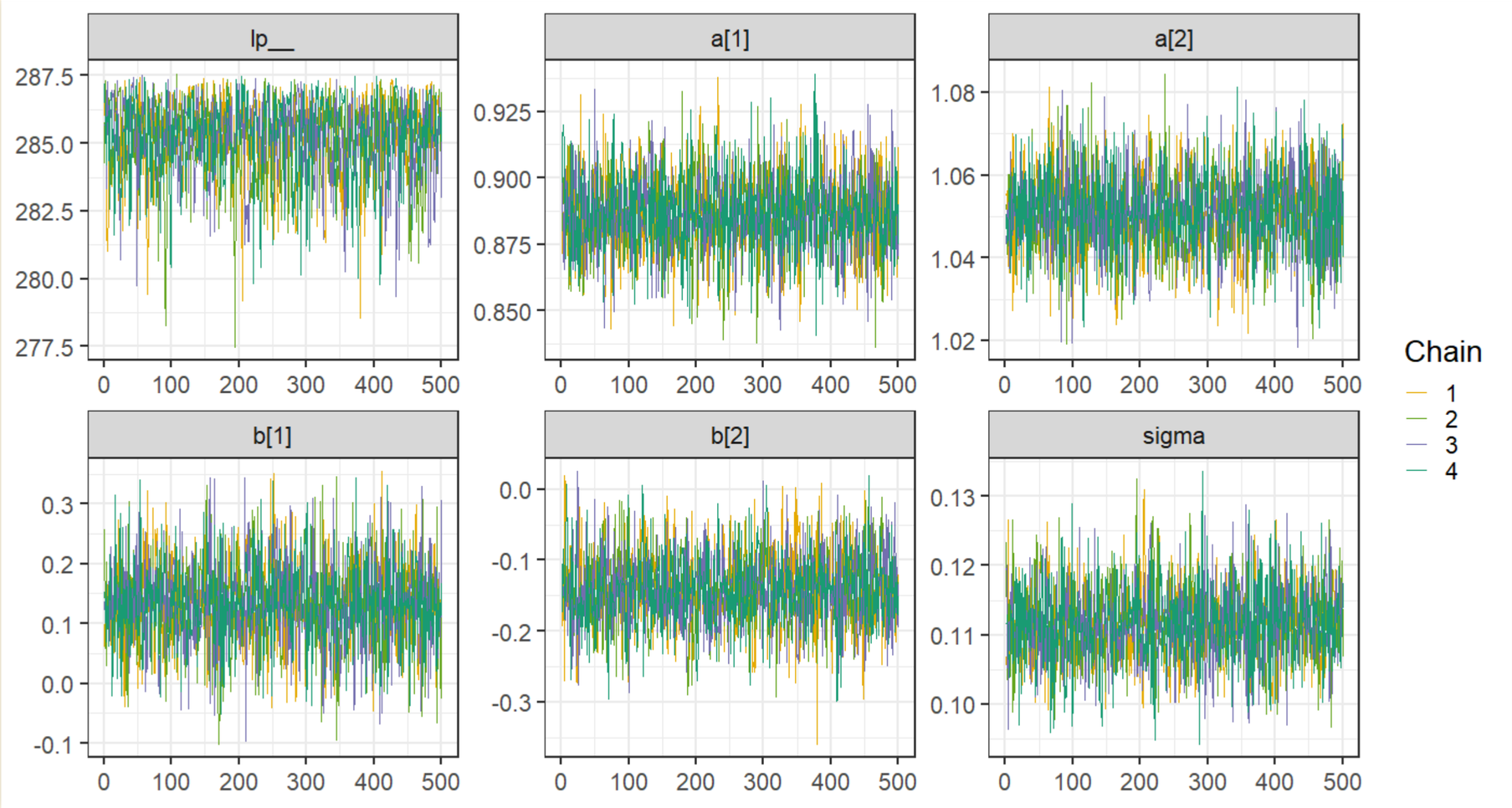
Pair-Correlation Plots

```
color_scheme_set("blue")
mcmc_pairs(mdl_draws, off_diag_args = list(size = 1, alpha = 0.05))
```



MCMC Trace Plots

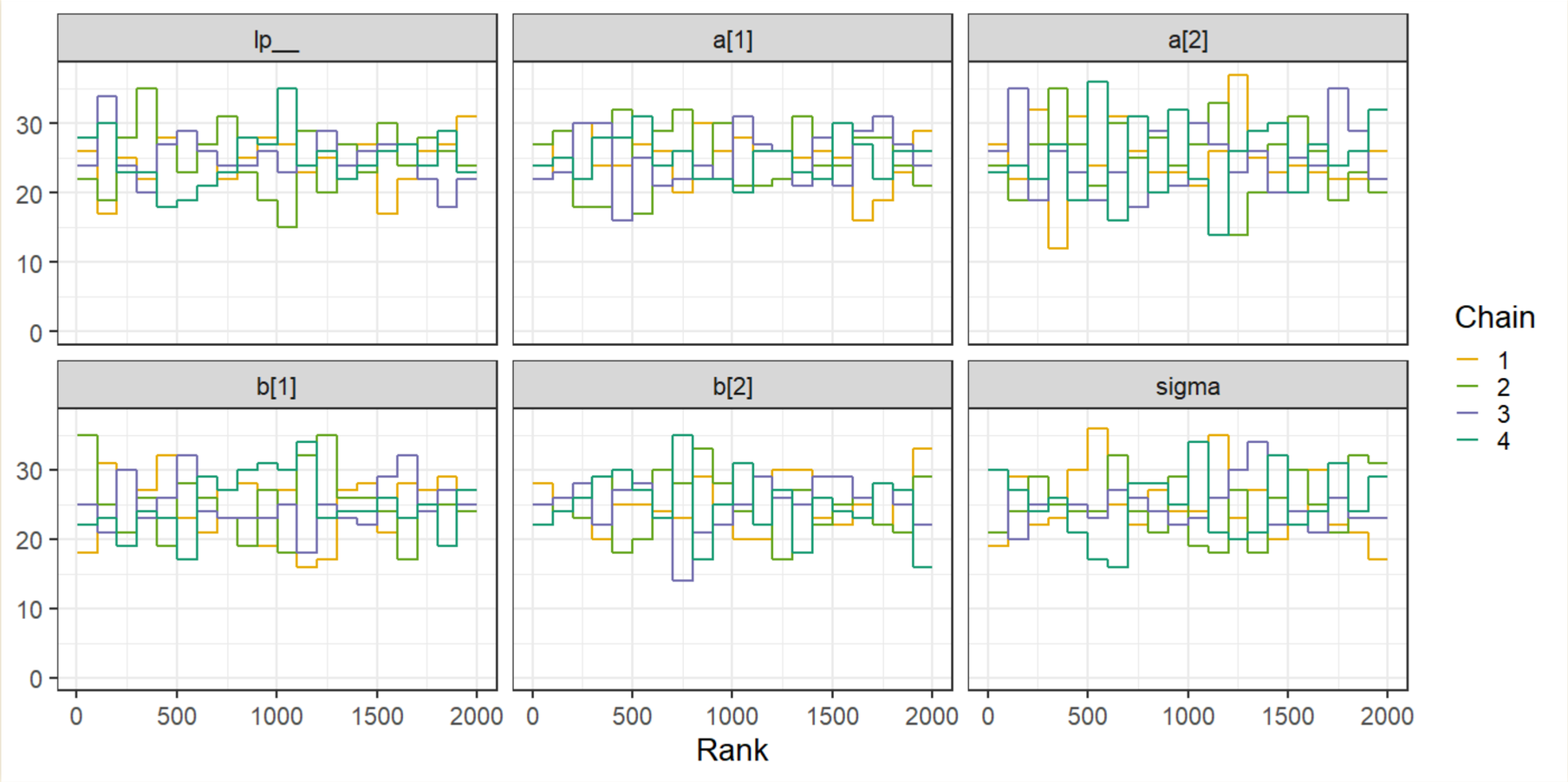
```
color_scheme_set("brewer-Dark2")
mcmc_trace(mdl_draws)
```



- Values of samples from each chain.
  - Should overlap nicely

MCMC Trace-Rank Plots

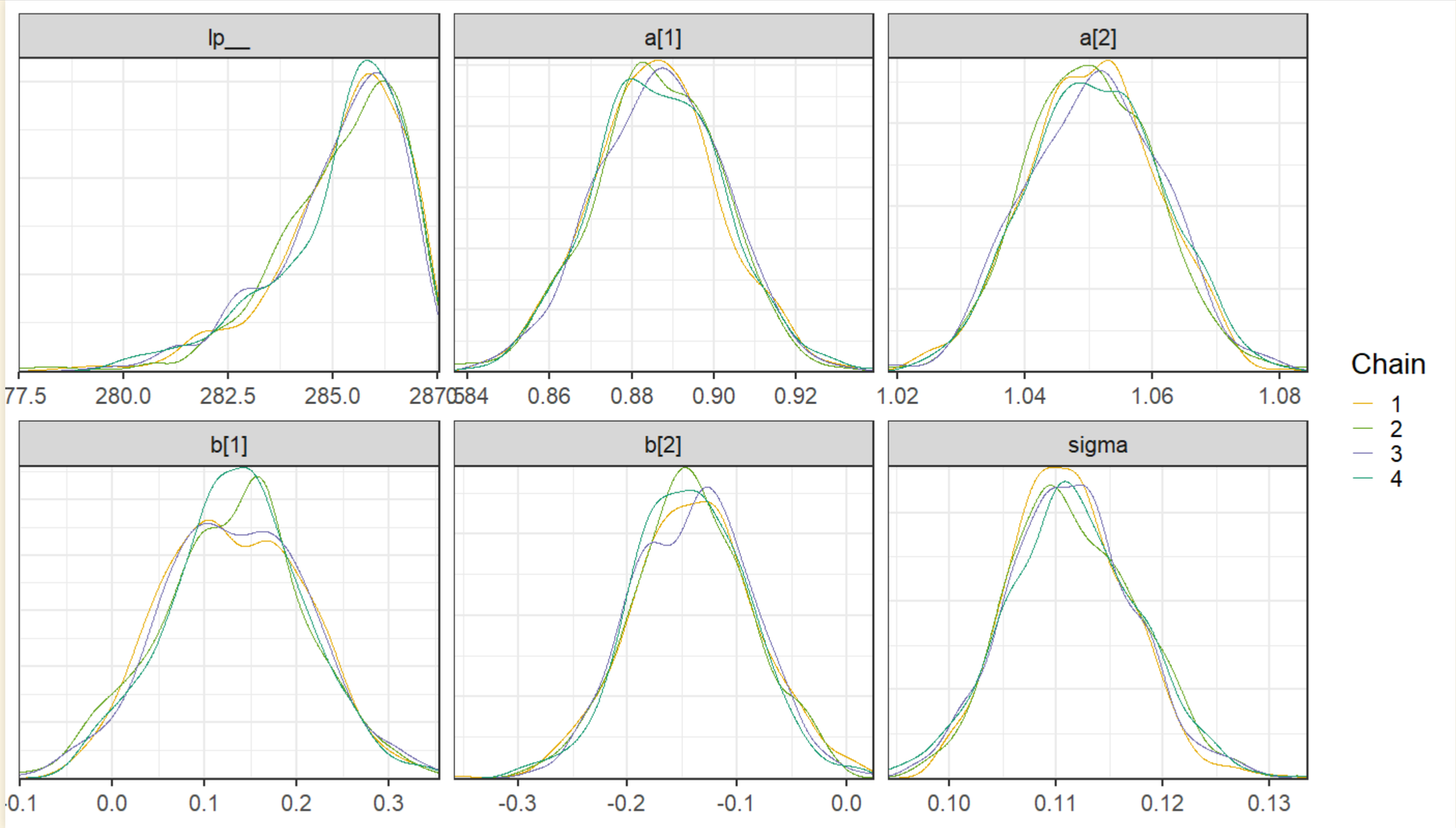
```
mcmc_rank_overlay(mdl_draws)
```



- Rank samples from smallest (1) to largest (2000).
  - Plot histogram of how many samples from each chain falls into each bin.
  - Easier to read than trace plots
  - Histograms for all four chains should look similar

Posterior Density Plots

```
mcmc_dens_overlay(mdl_draws)
```



MCMC Problems

Poorly Fit Model

```
y <- c(-1, 1)
set.seed(11)
mdl_bad <- ulam(
  alist(
    y ~ dnorm(mu, sigma),
    mu <- alpha,
    alpha ~ dnorm(0, 1000),
    sigma ~ dexp(0.0001)
  ), data = list(y = y), chains = 4, cores = 4 )
```

- 2 data observations
- Very uninformative priors
- Gives warning:

```
Warning: 87 of 2000 (4.0%) transitions ended with a
divergence.
See https://mc-stan.org/misc/warnings for details.
```

```
show(mdl_bad)
```

```
## Hamiltonian Monte Carlo approximation
## 2000 samples from 4 chains
##
## Sampling durations (seconds):
##   chain_id warmup sampling total
## 1        1   0.10     0.02  0.13
## 2        2   0.05     0.02  0.07
## 3        3   0.03     0.02  0.05
## 4        4   0.04     0.03  0.07
##
## Formula:
## y ~ dnorm(mu, sigma)
## mu <- alpha
## alpha ~ dnorm(0, 1000)
## sigma ~ dexp(1e-04)
```

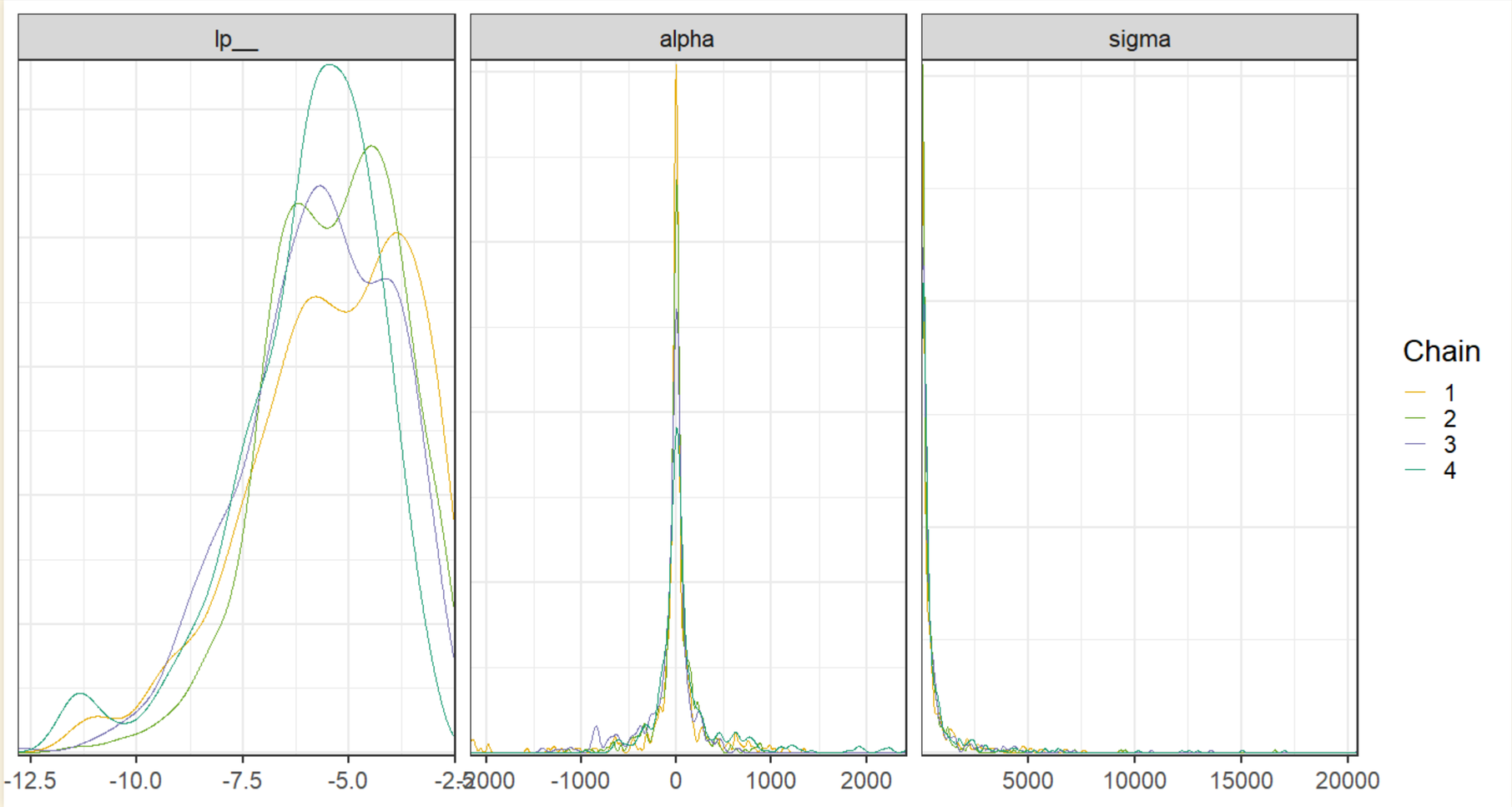
```
precis_show(precis(mdl_bad, digits=2))
```

```
##         mean      sd   5.5%   94.5% rhat ess_bulk
## alpha  -6.01  339.32 -446.5  415.13 1.01   231.84
## sigma 560.05 1436.09   19.6 2285.11 1.02   121.58
```

- `ess_bulk`: <200 effective samples out of 2000.
- `sd`: Huge standard deviations in posteriors.

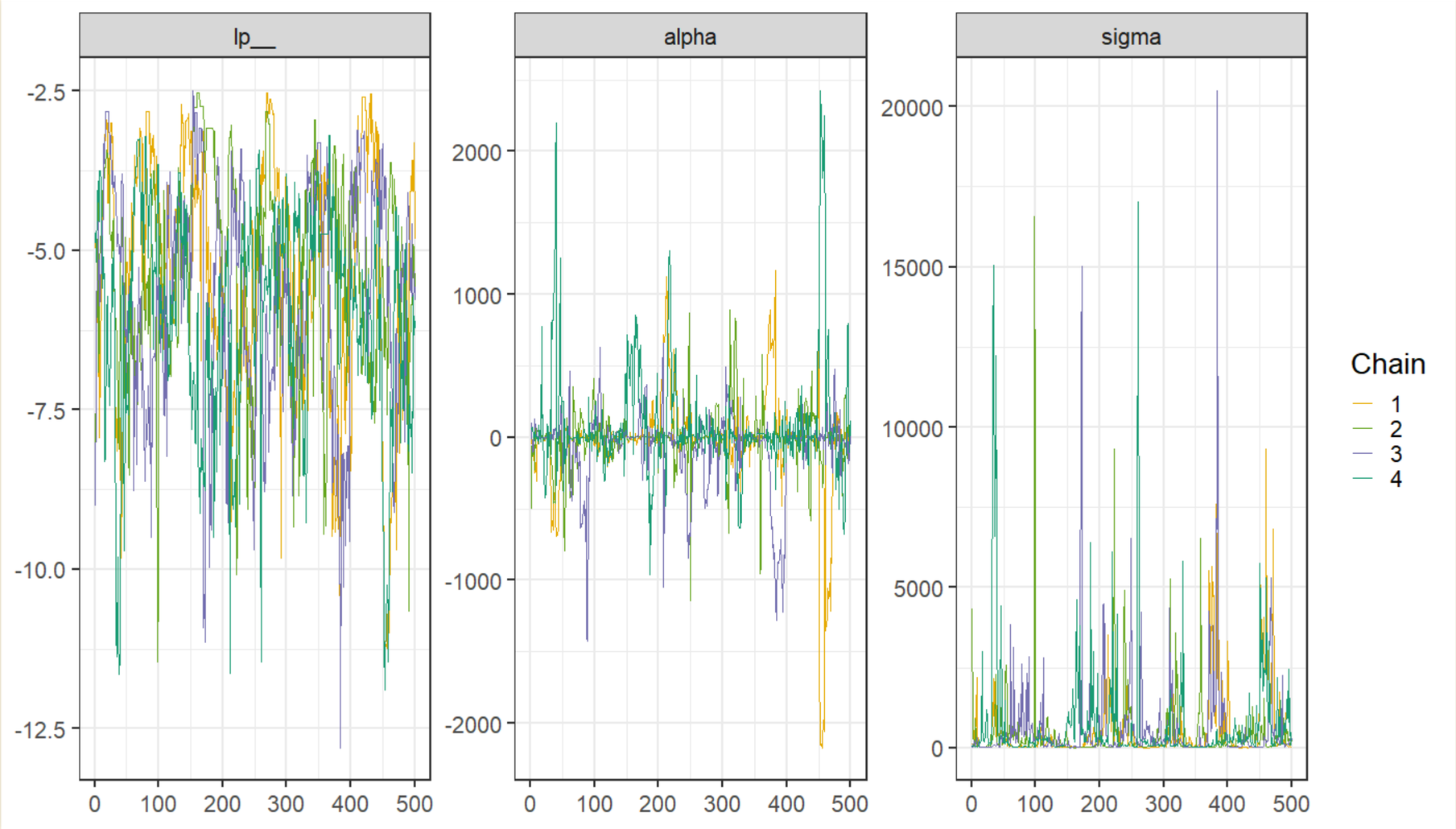
Show Posteriors

```
bad_mdl_fit <- mdl_bad@cstanfit
bad_mdl_draws <- bad_mdl_fit$draws()
color_scheme_set("brewer-Dark2")
mcmc_dens_overlay(bad_mdl_draws)
```



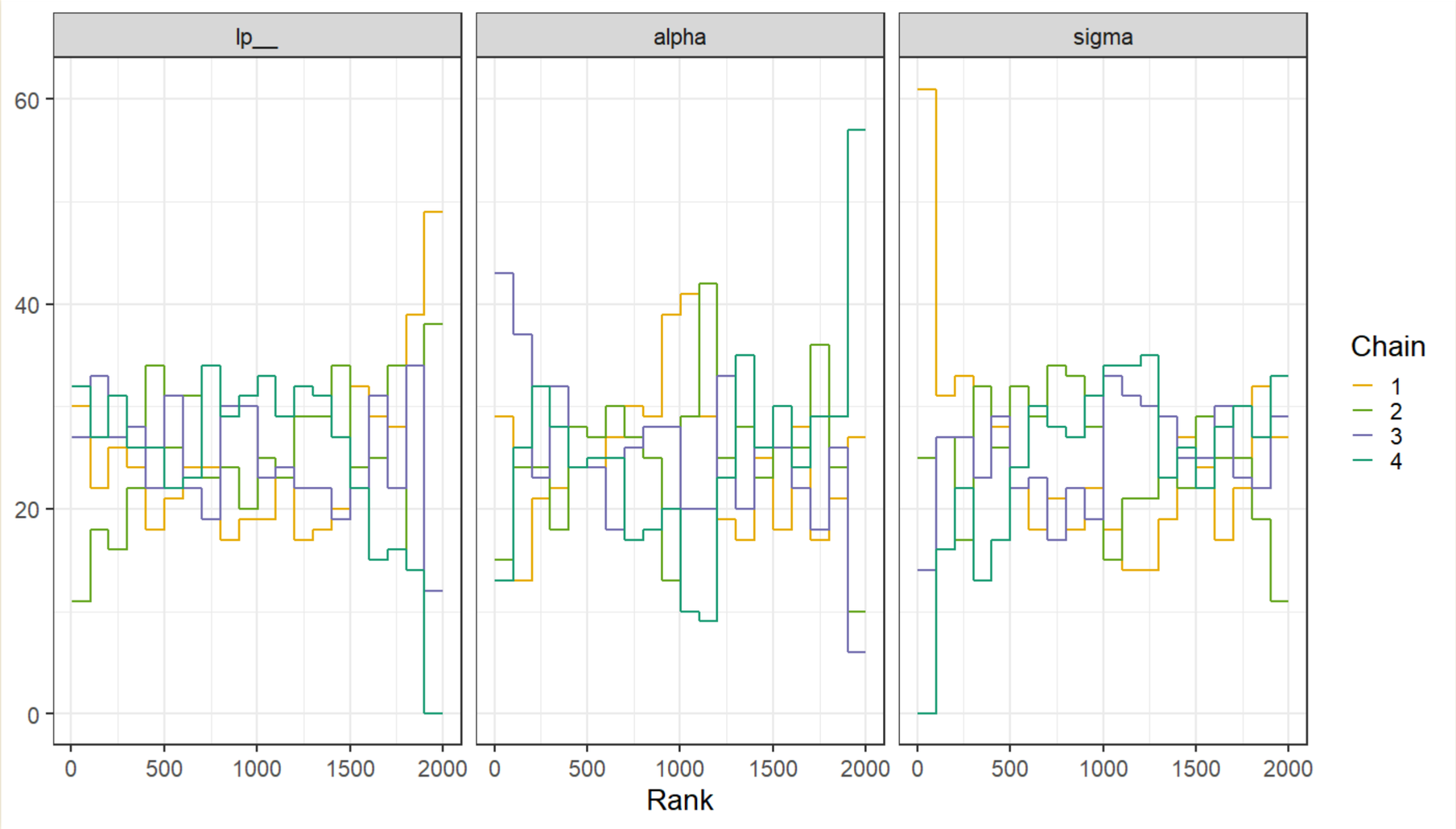
MCMC Trace Plots

```
mcmc_trace(bad_mdl_draws)
```



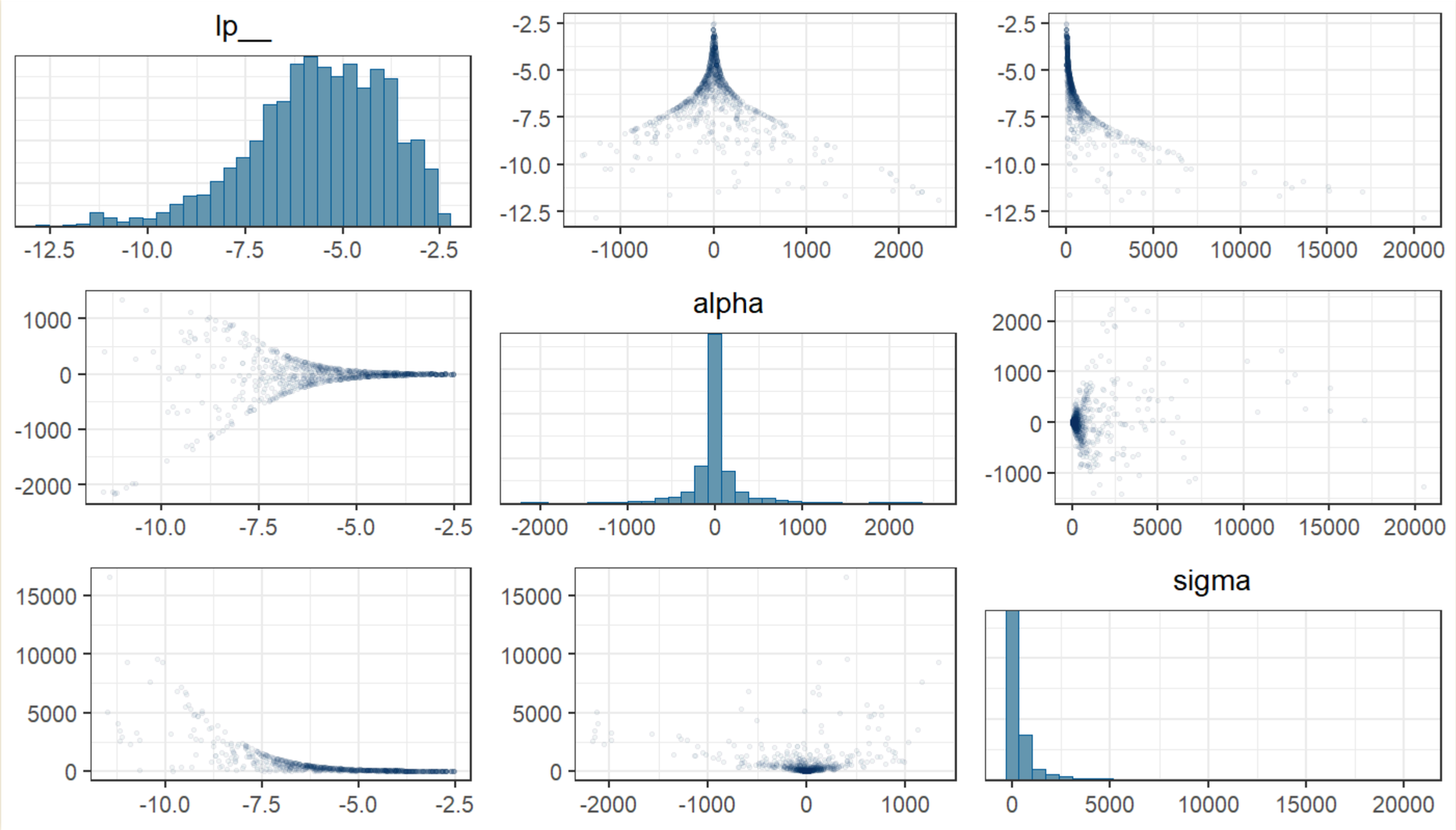
MCMC Trace-Rank Plots

```
mcmc_rank_overlay(bad_mdl_draws)
```



Pair-Correlation Plots: Funnels

```
color_scheme_set("blue")
mcmc_pairs(bad_mdl_draws, off_diag_args = list(size = 1, alpha = 0.05))
```



Better Model

Better Model

```
set.seed(11)
mdl_better <- ulam(
  alist(
    y ~ dnorm(mu, sigma),
    mu <- alpha,
    alpha ~ dnorm(0, 10),
    sigma ~ dexp(1)
  ), data = list(y = y), chains = 4, cores = 4 )
```

- Replace uninformative priors with weakly informative priors.
  - Weakly informative priors help a lot.

```
show(mdl_better)
```

```
## Hamiltonian Monte Carlo approximation
## 2000 samples from 4 chains
##
## Sampling durations (seconds):
##   chain_id warmup sampling total
## 1        1   0.03     0.02  0.05
## 2        2   0.02     0.02  0.04
## 3        3   0.02     0.02  0.04
## 4        4   0.03     0.02  0.05
##
## Formula:
## y ~ dnorm(mu, sigma)
## mu <- alpha
## alpha ~ dnorm(0, 10)
## sigma ~ dexp(1)
```

```
precis_show(precis(mdl_better, digits=2))
```

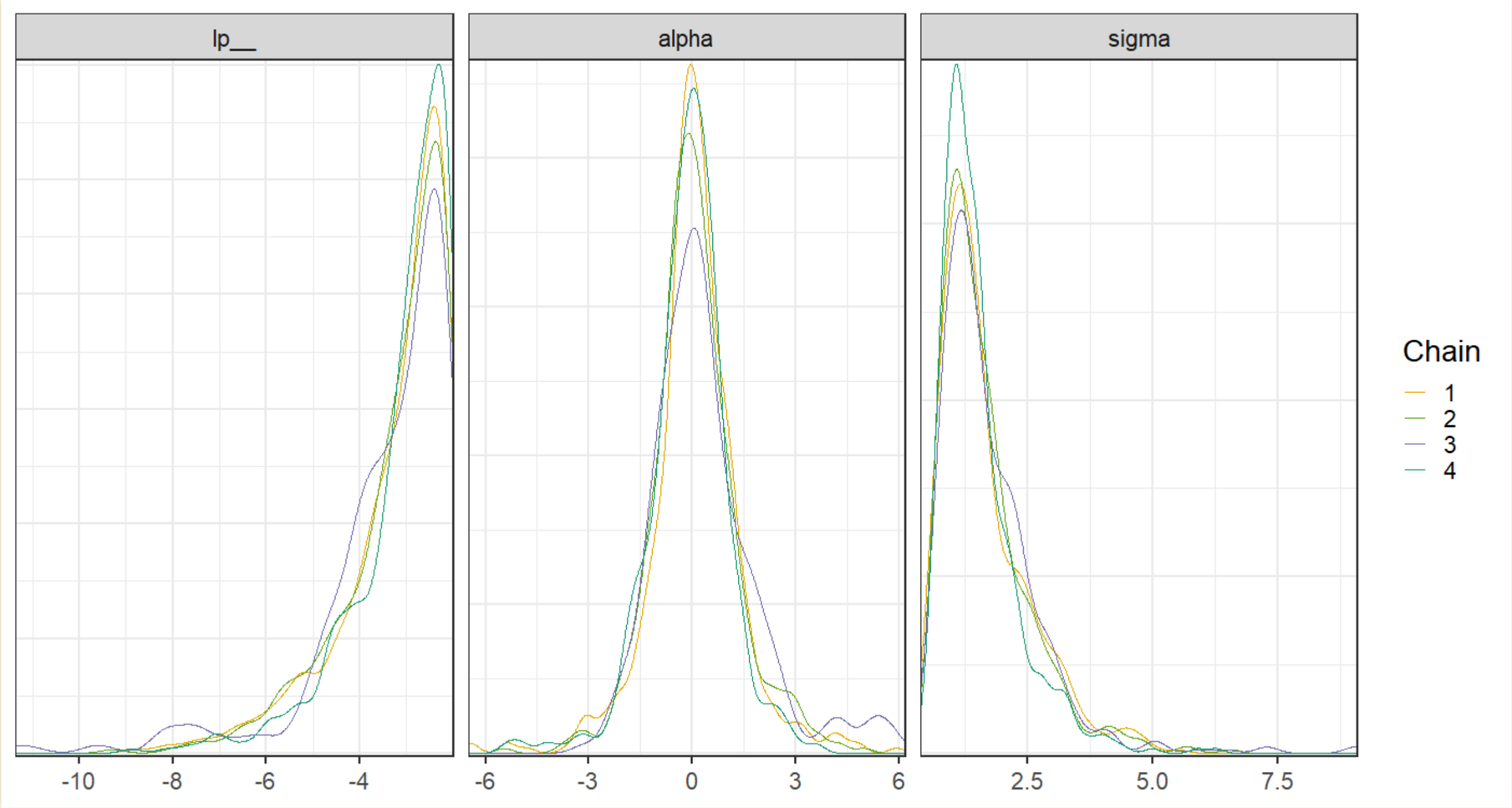
```
##       mean   sd  5.5% 94.5% rhat ess_bulk
## alpha 0.10 1.35 -1.79  2.26 1.01   494.79
## sigma 1.62 0.91  0.68  3.22 1.01   485.01
```

- Even weakly informative priors help a lot!
- `ess_bulk`: almost 500 effective samples.
- `sd`: Much smaller.

Show Posteriors

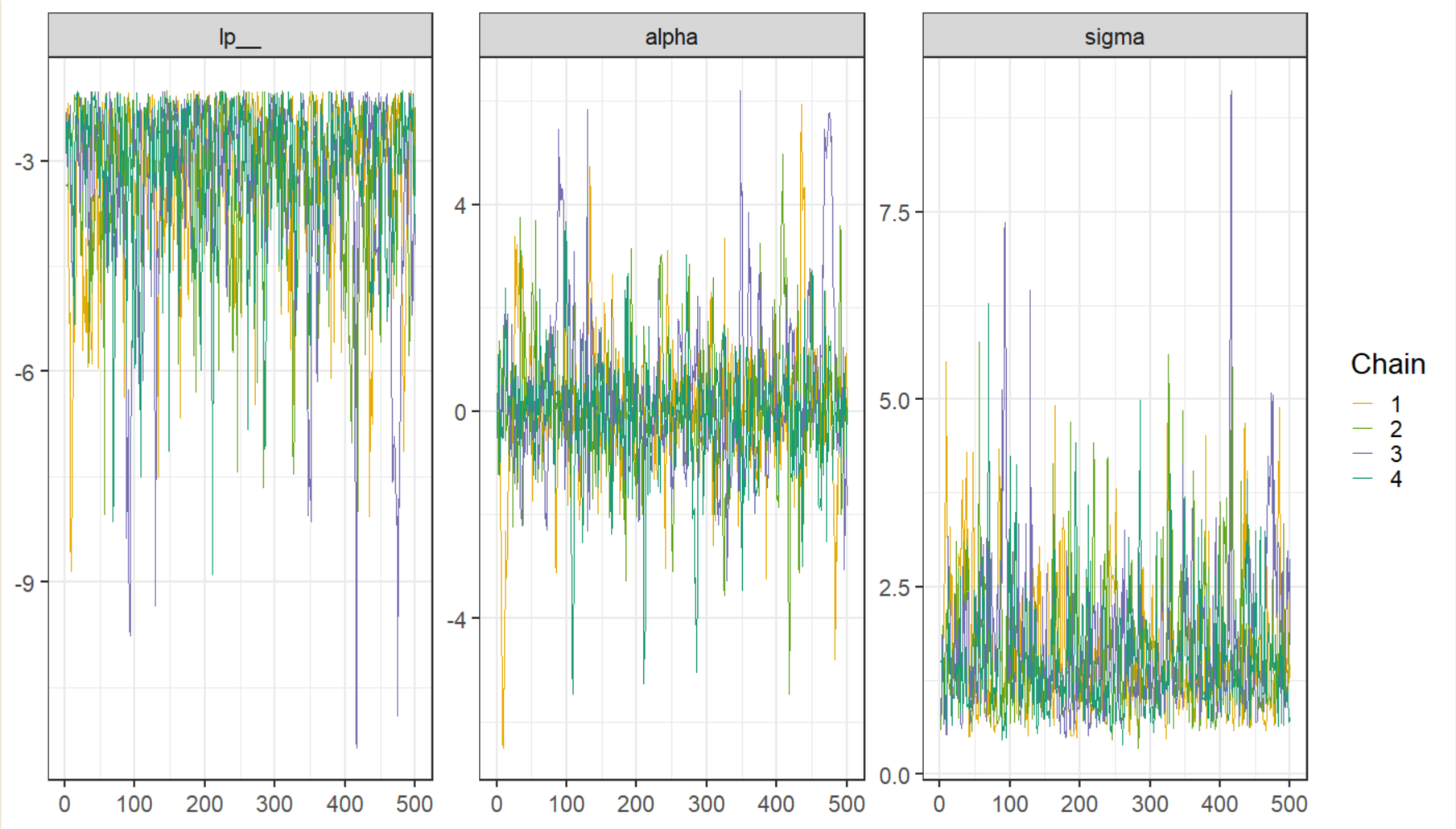
```
better_mdl_fit <- mdl_better@cstanfit
better_mdl_draws <- better_mdl_fit$draws()
```

```
color_scheme_set("brewer-Dark2")
mcmc_dens_overlay(better_mdl_draws)
```



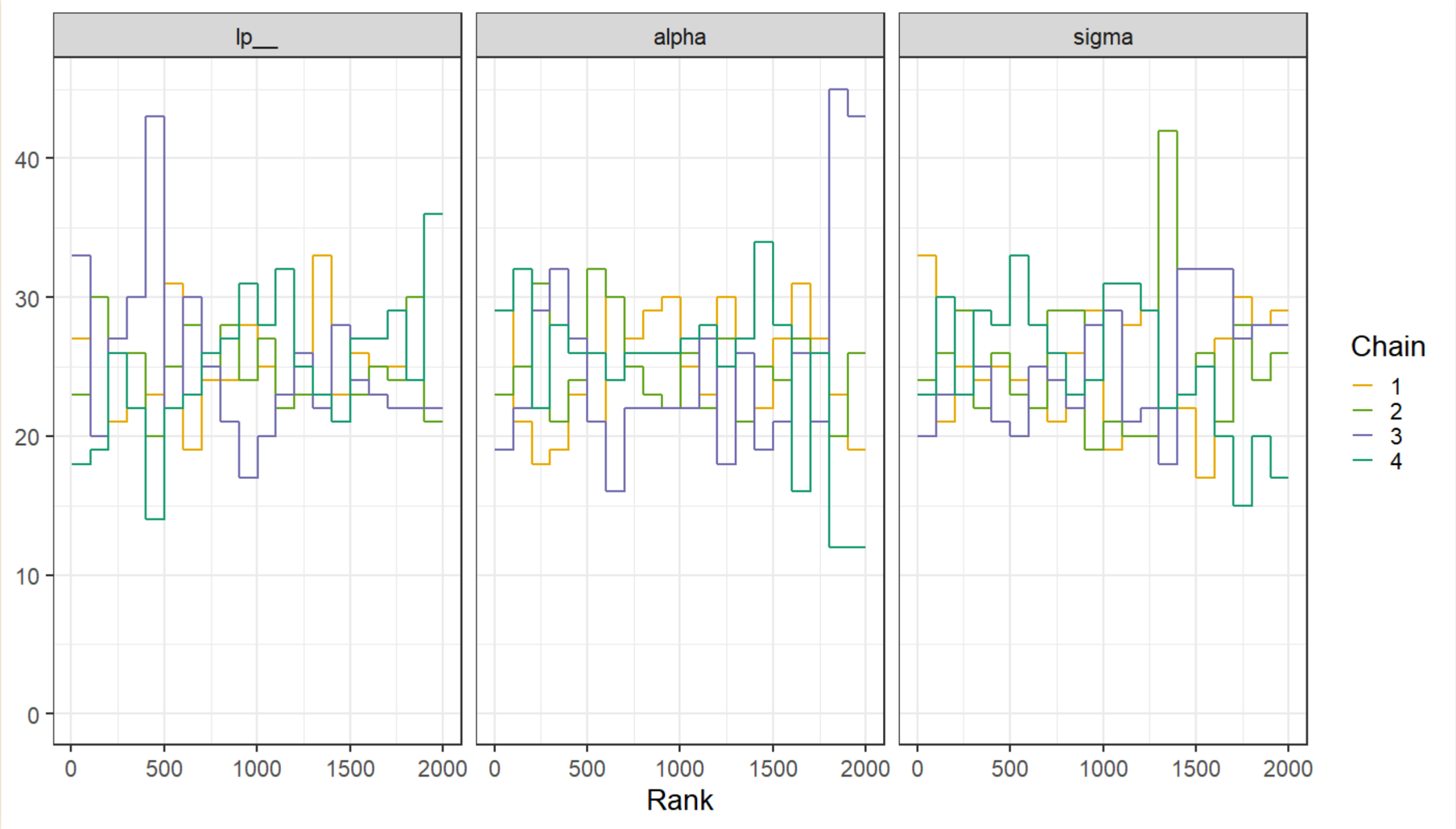
MCMC Trace Plots

```
mcmc_trace(better_mdl_draws)
```



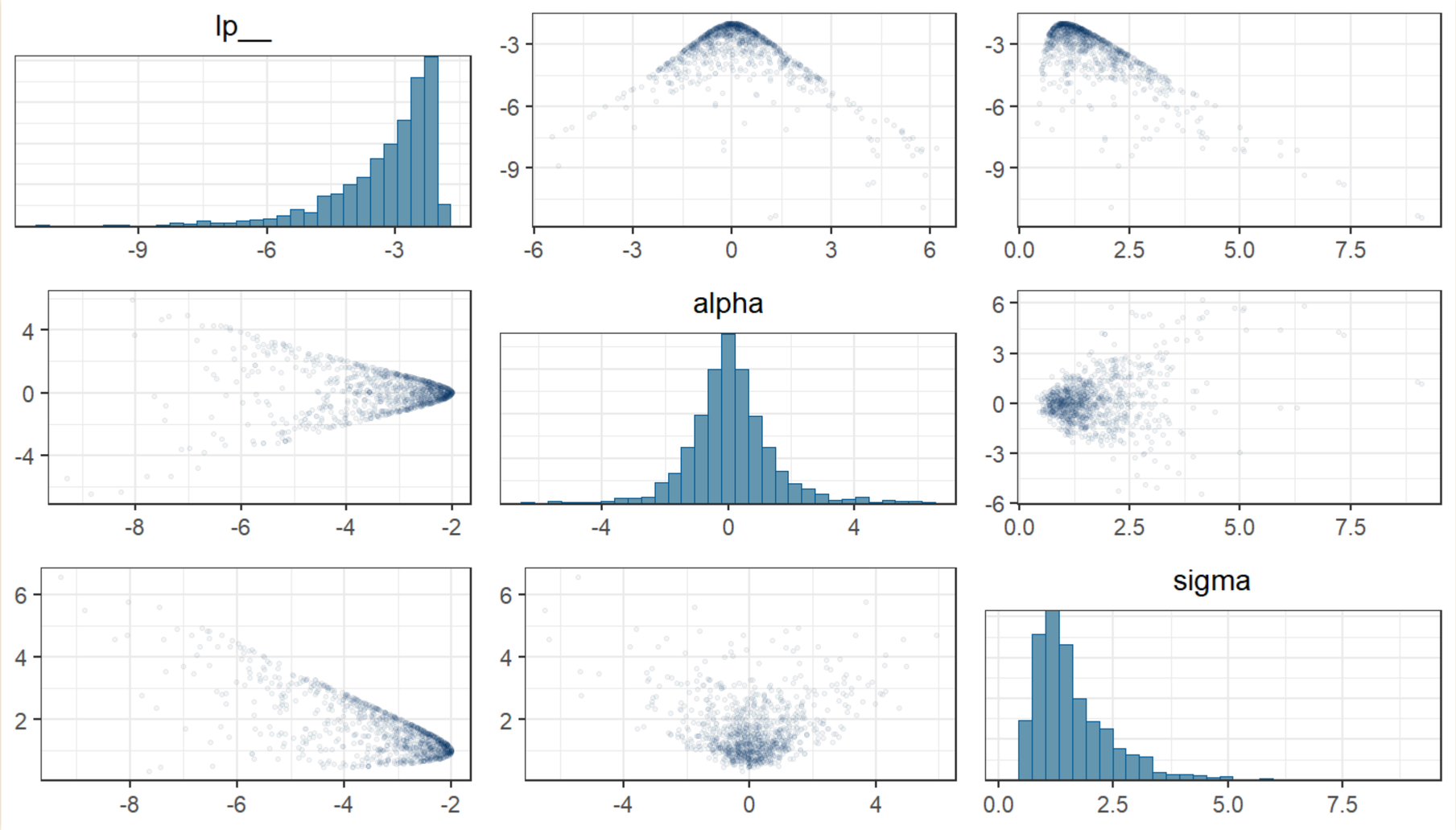
MCMC Trace-Rank Plots

```
mcmc_rank_overlay(better_mdl_draws)
```



Pair-Correlation Plots

```
color_scheme_set("blue")
mcmc_pairs(better_mdl_draws, off_diag_args = list(size = 1, alpha = 0.05))
```



Non-Identifiable Models

Non-Identifiable Model

```
set.seed(41)
y <- rnorm(100, mean = 0, sd = 1)
```

- Model

$$y \sim \text{Normal}(\mu, \sigma)$$
$$\mu = \alpha_1 + \alpha_2$$
$$\alpha_1 \sim \text{Normal}(0, 1000)$$
$$\alpha_2 \sim \text{Normal}(0, 1000)$$
$$\sigma \sim \text{Exponential}(1)$$

  - Why is this non-identifiable?

```
set.seed(11)
mdl_non_id <- ulam(
  alist(
    y ~ dnorm(mu, sigma),
    mu <- a1 + a2,
    a1 ~ dnorm(0, 1000),
    a2 ~ dnorm(0, 1000),
    sigma ~ dexp(1)
  ), data = list(y = y), chains = 4, cores = 4 )
```

- Error message:

```
Warning: 1522 of 2000 (76.0%) transitions hit the
maximum treedepth limit of 10.
See https://mc-stan.org/misc/warnings for details.
```

```
show(mdl_non_id)
```

```
## Hamiltonian Monte Carlo approximation
## 2000 samples from 4 chains
##
## Sampling durations (seconds):
##   chain_id warmup sampling total
## 1        1   1.07     1.11  2.19
## 2        2   1.09     1.09  2.18
## 3        3   0.98     1.16  2.14
## 4        4   1.05     1.15  2.20
##
## Formula:
## y ~ dnorm(mu, sigma)
## mu <- a1 + a2
## a1 ~ dnorm(0, 1000)
## a2 ~ dnorm(0, 1000)
## sigma ~ dexp(1)
```

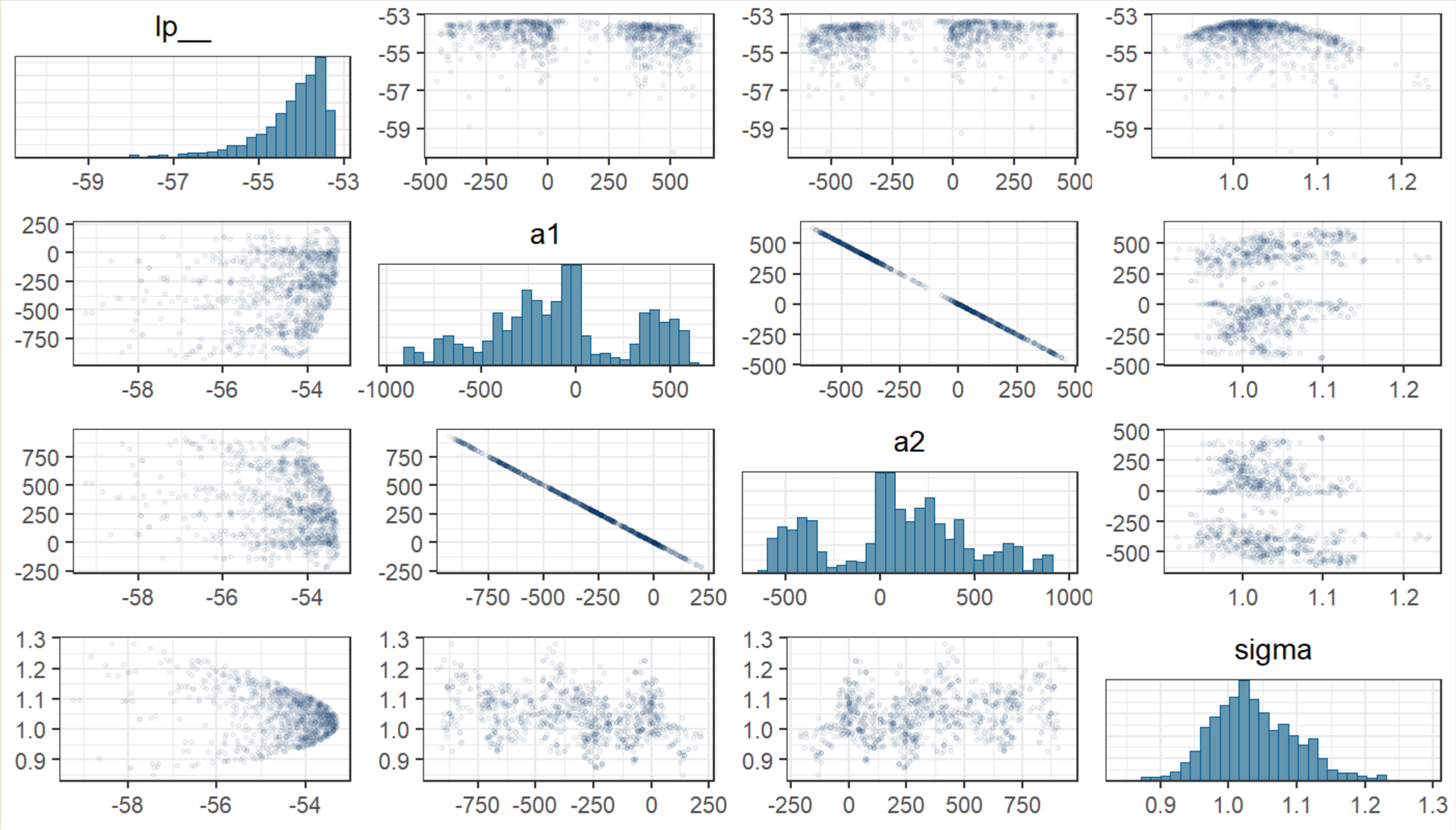
```
precis_show(precis(mdl_non_id, digits=2))
```

```
##         mean     sd    5.5%  94.5% rhat ess_bulk
## a1    -84.92 374.08 -702.21 519.46 2.36     4.99
## a2     85.11 374.08 -519.34 702.33 2.36     4.99
## sigma   1.04   0.06    0.95   1.14 1.06    89.58
```

Pair-Correlation Plots

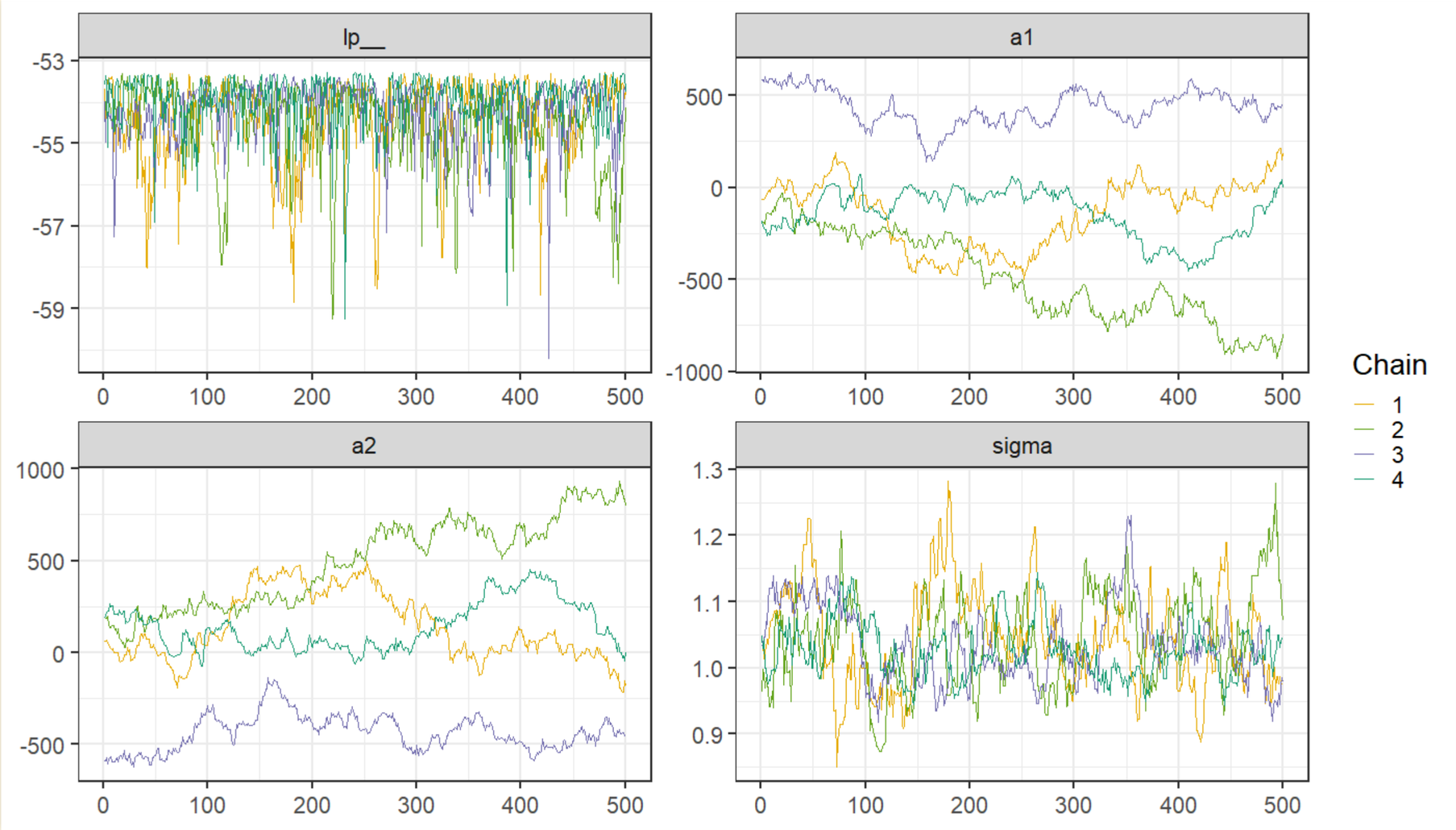
```
nid_mdl_fit <- mdl_non_id@cstanfit
nid_mdl_draws <- nid_mdl_fit$draws()

color_scheme_set("blue")
mcmc_pairs(nid_mdl_draws, off_diag_args = list(size = 1, alpha = 0.05))
```



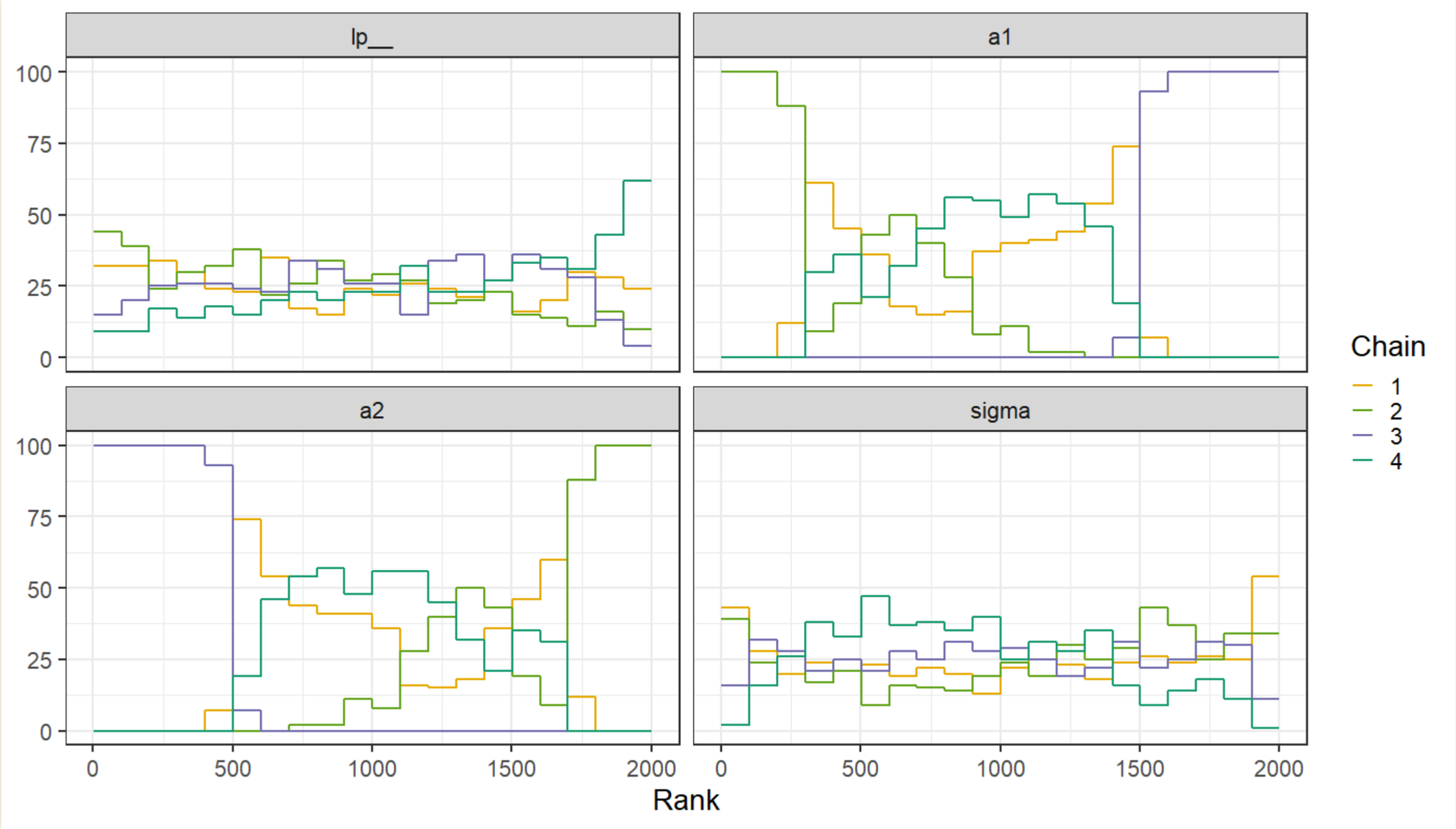
MCMC Trace Plots

```
color_scheme_set("brewer-Dark2")
mcmc_trace(nid_mdl_draws)
```



MCMC Trace-Rank Plots

```
mcmc_rank_overlay(nid_mdl_draws)
```



Regularize Model

Regularize Model

- Model with regularizing priors

$$
\begin{aligned}
y &\sim \text{Normal}(\mu, \sigma) \\
\mu &= \alpha_1 + \alpha_2 \\
\alpha_1 &\sim \text{Normal}(0, 10) \\
\alpha_2 &\sim \text{Normal}(0, 10) \\
\sigma &\sim \text{Exponential}(1)
\end{aligned}
$$

```
set.seed(11)
mdl_reg <- ulam(
  alist(
    y ~ dnorm(mu, sigma),
    mu <- a1 + a2,
    a1 ~ dnorm(0, 10),
    a2 ~ dnorm(0, 10),
    sigma ~ dexp(1)
  ), data = list(y = y), chains = 4, cores = 4 )
```

- Error message:

```
Warning: 14 of 2000 (1.0%) transitions hit the maximum
treedepth limit of 10.
See https://mc-stan.org/misc/warnings for details.
```

```
show(mdl_reg)
```

```
## Hamiltonian Monte Carlo approximation
## 2000 samples from 4 chains
##
## Sampling durations (seconds):
##   chain_id warmup sampling total
## 1        1   0.38     0.44  0.82
## 2        2   0.36     0.56  0.91
## 3        3   0.40     0.37  0.77
## 4        4   0.38     0.40  0.78
##
## Formula:
## y ~ dnorm(mu, sigma)
## mu <- a1 + a2
## a1 ~ dnorm(0, 10)
## a2 ~ dnorm(0, 10)
## sigma ~ dexp(1)
```

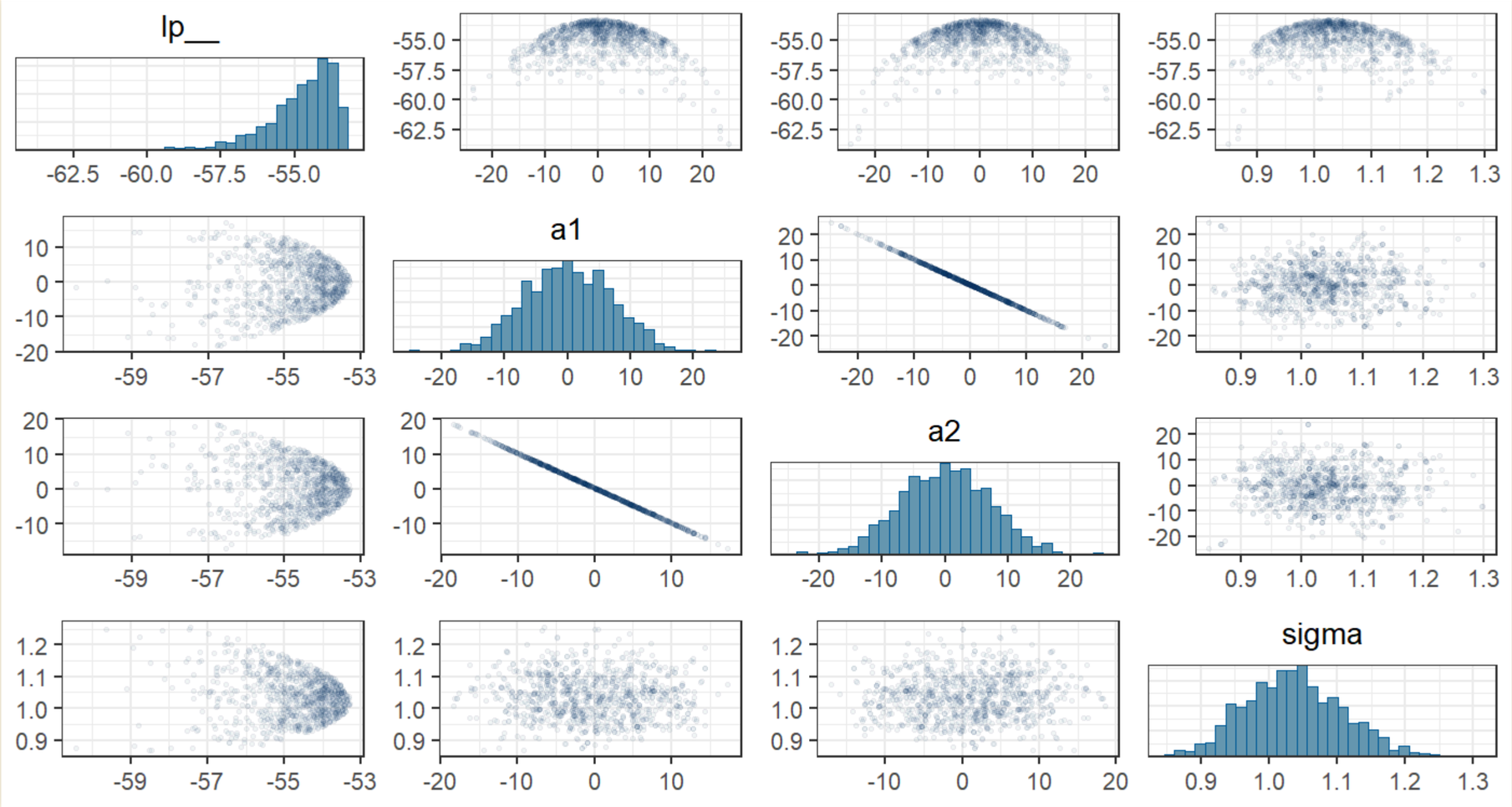
```
precis_show(precis(mdl_reg, digits=2))
```

```
##       mean   sd   5.5% 94.5% rhat ess_bulk
## a1    0.12 7.08 -11.01 11.63 1.01   543.50
## a2    0.06 7.08 -11.45 11.15 1.01   544.07
## sigma 1.04 0.07   0.93  1.16 1.00   762.89
```

Pair-Correlation Plots

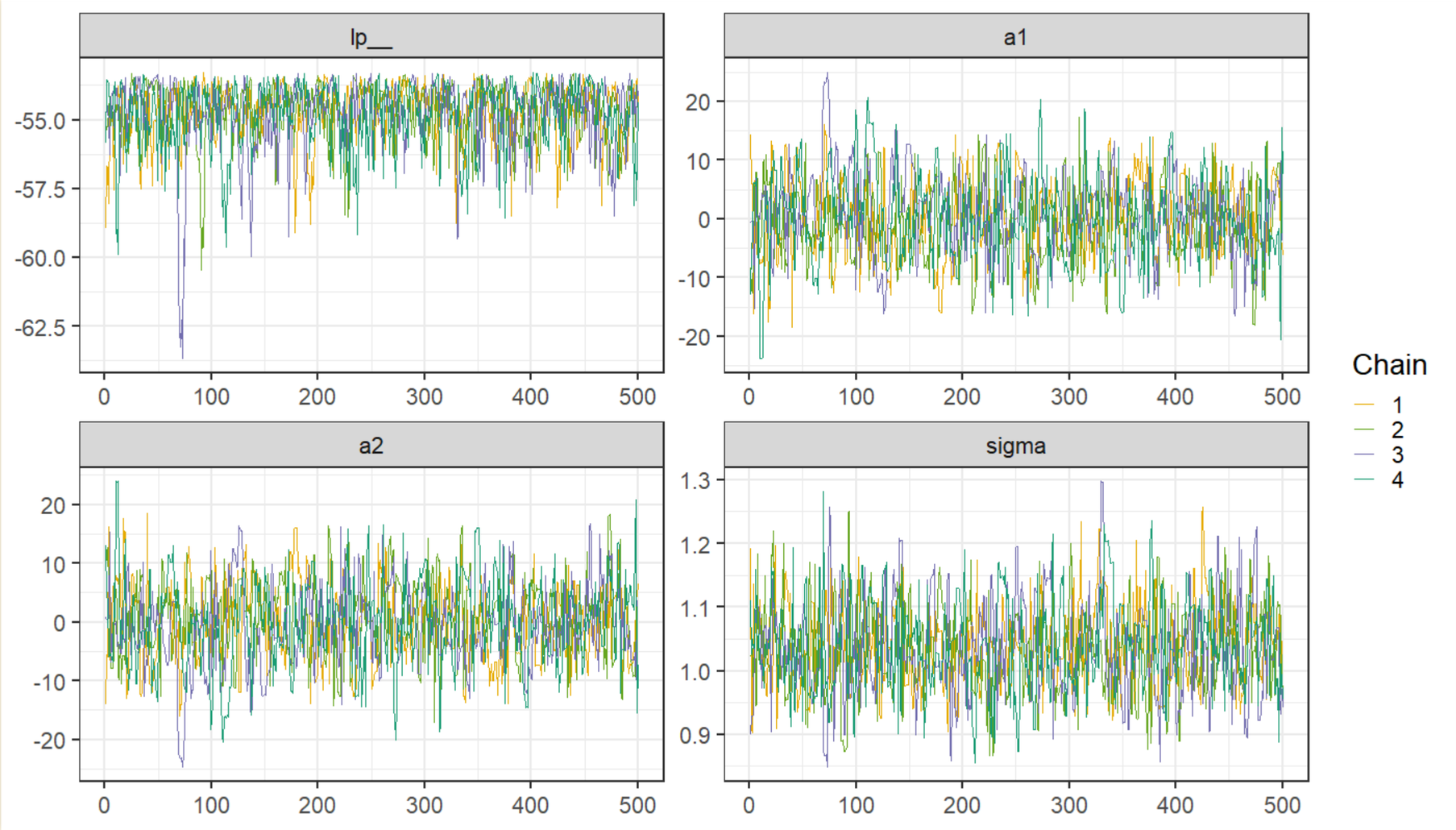
```
reg_mdl_fit <- mdl_reg@cstanfit
reg_mdl_draws <- reg_mdl_fit$draws()

color_scheme_set("blue")
mcmc_pairs(reg_mdl_draws, off_diag_args = list(size = 1, alpha = 0.05))
```



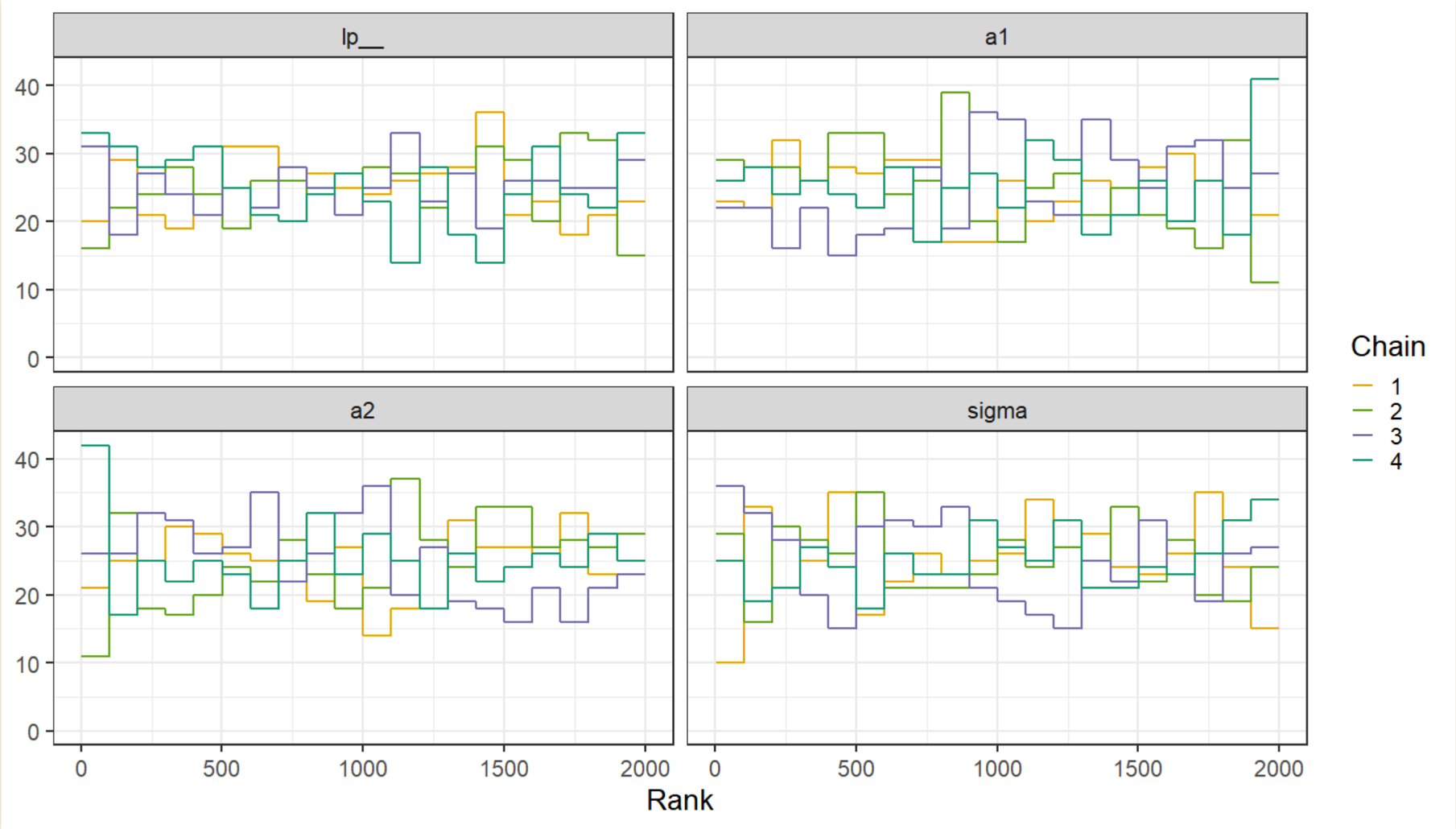
MCMC Trace Plots

```
color_scheme_set("brewer-Dark2")
mcmc_trace(reg_mdl_draws)
```



MCMC Trace-Rank Plots

```
mcmc_rank_overlay(reg_mdl_draws)
```



Make Model Identifiable

Change Variables

- Model with regularizing priors

$$y \sim \text{Normal}(\mu, \sigma)$$
$$\mu = \alpha$$
$$\alpha \sim \text{Normal}(0, 10)$$
$$\sigma \sim \text{Exponential}(1)$$

```
set.seed(11)
mdl_re_id <- ulam(
  alist(
    y ~ dnorm(mu, sigma),
    mu <- a,
    a ~ dnorm(0, 10),
    sigma ~ dexp(1)
  ), data = list(y = y), chains = 4, cores = 4 )
```

```
show(mdl_re_id)
```

```
## Hamiltonian Monte Carlo approximation
## 2000 samples from 4 chains
##
## Sampling durations (seconds):
##   chain_id warmup sampling total
## 1        1   0.02     0.01  0.03
## 2        2   0.01     0.01  0.03
## 3        3   0.01     0.01  0.03
## 4        4   0.02     0.01  0.03
##
## Formula:
## y ~ dnorm(mu, sigma)
## mu <- a
## a ~ dnorm(0, 10)
## sigma ~ dexp(1)
```

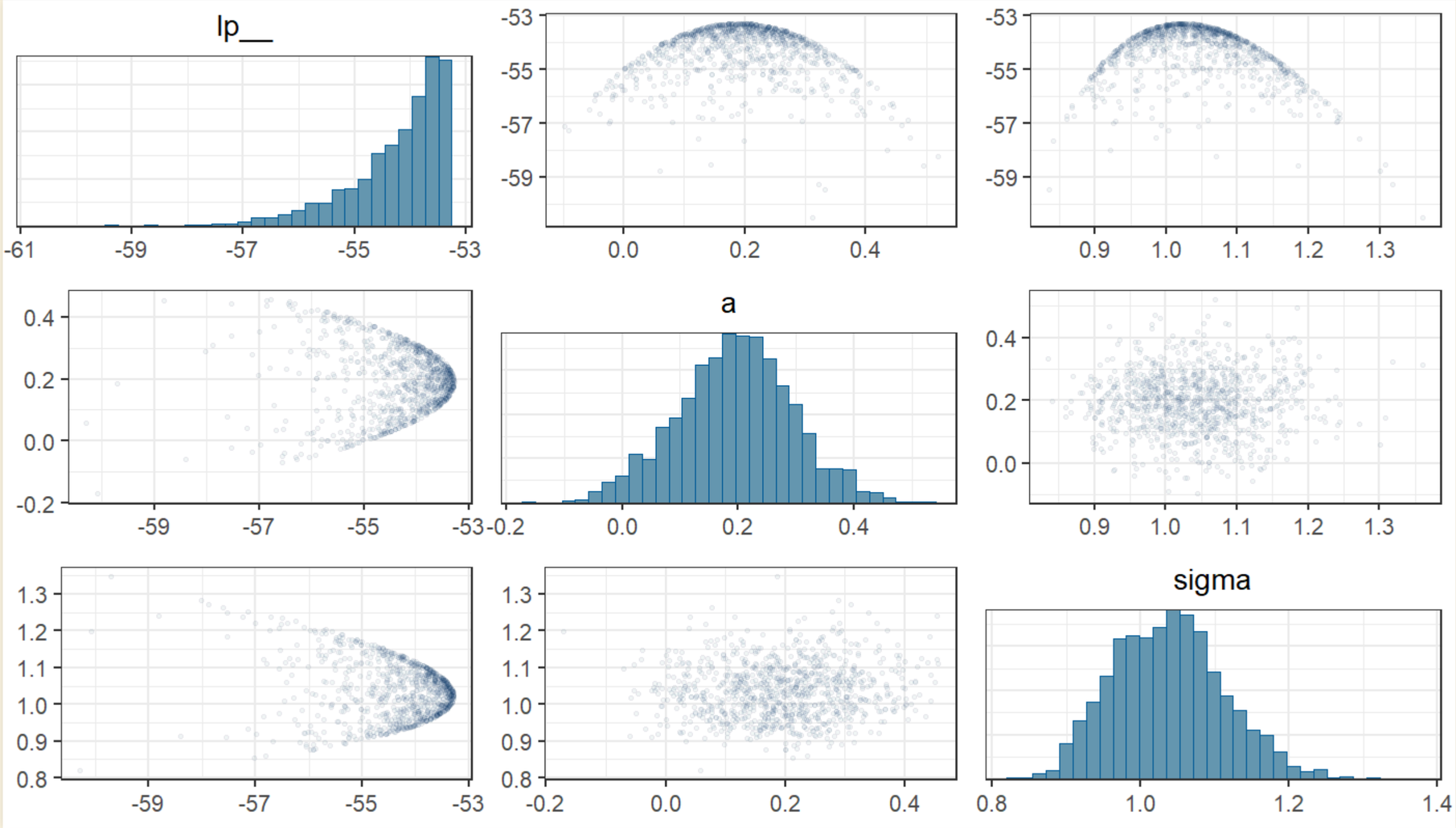
```
precis_show(precis(mdl_re_id, digits=2))
```

```
##       mean   sd 5.5% 94.5% rhat ess_bulk
## a     0.19 0.10 0.03  0.36    1  1439.69
## sigma 1.04 0.08 0.92  1.16    1  1444.62
```

Pair-Correlation Plots

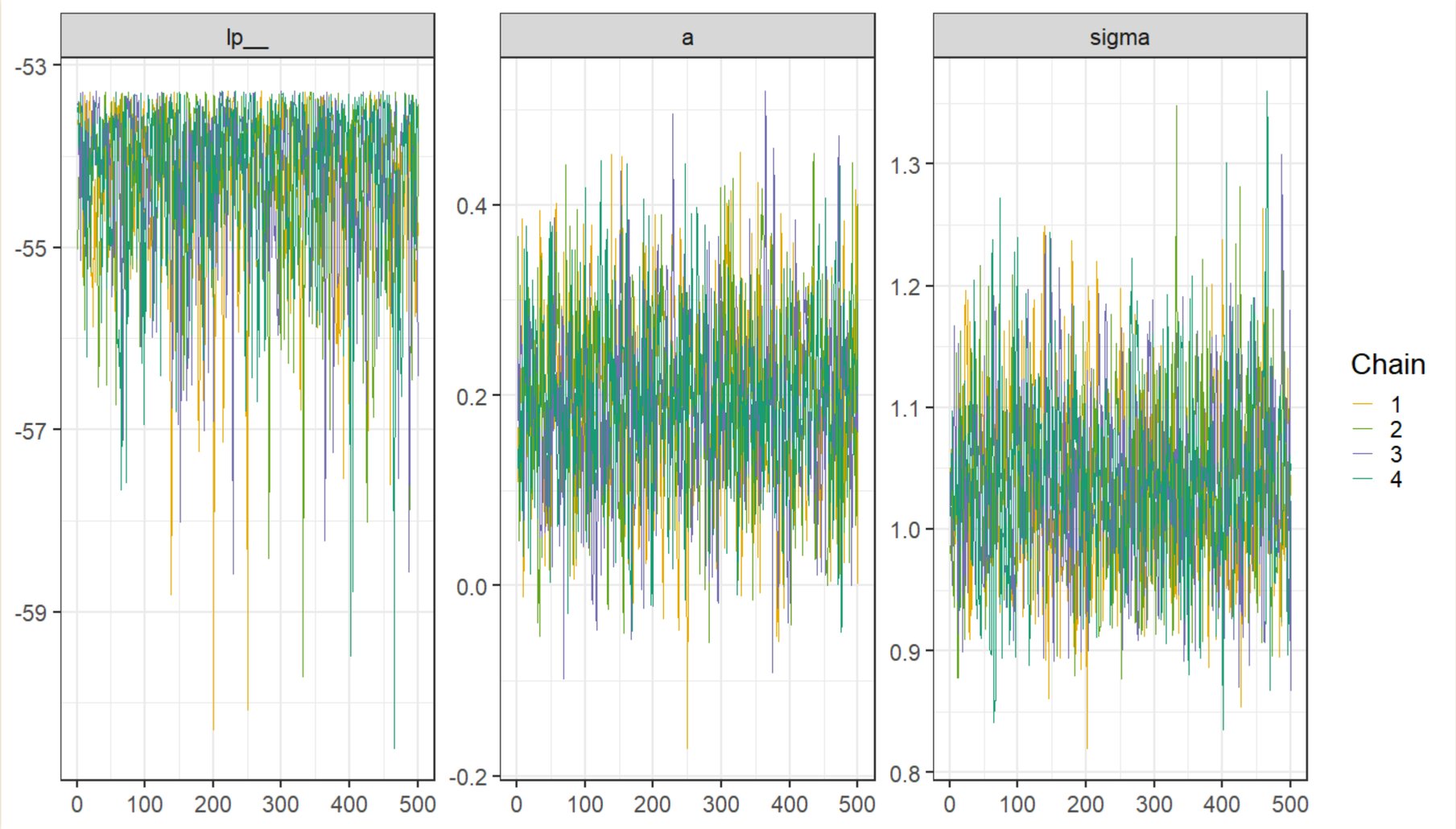
```
re_id_mdl_fit <- mdl_re_id@cstanfit
re_id_mdl_draws <- re_id_mdl_fit$draws()
```

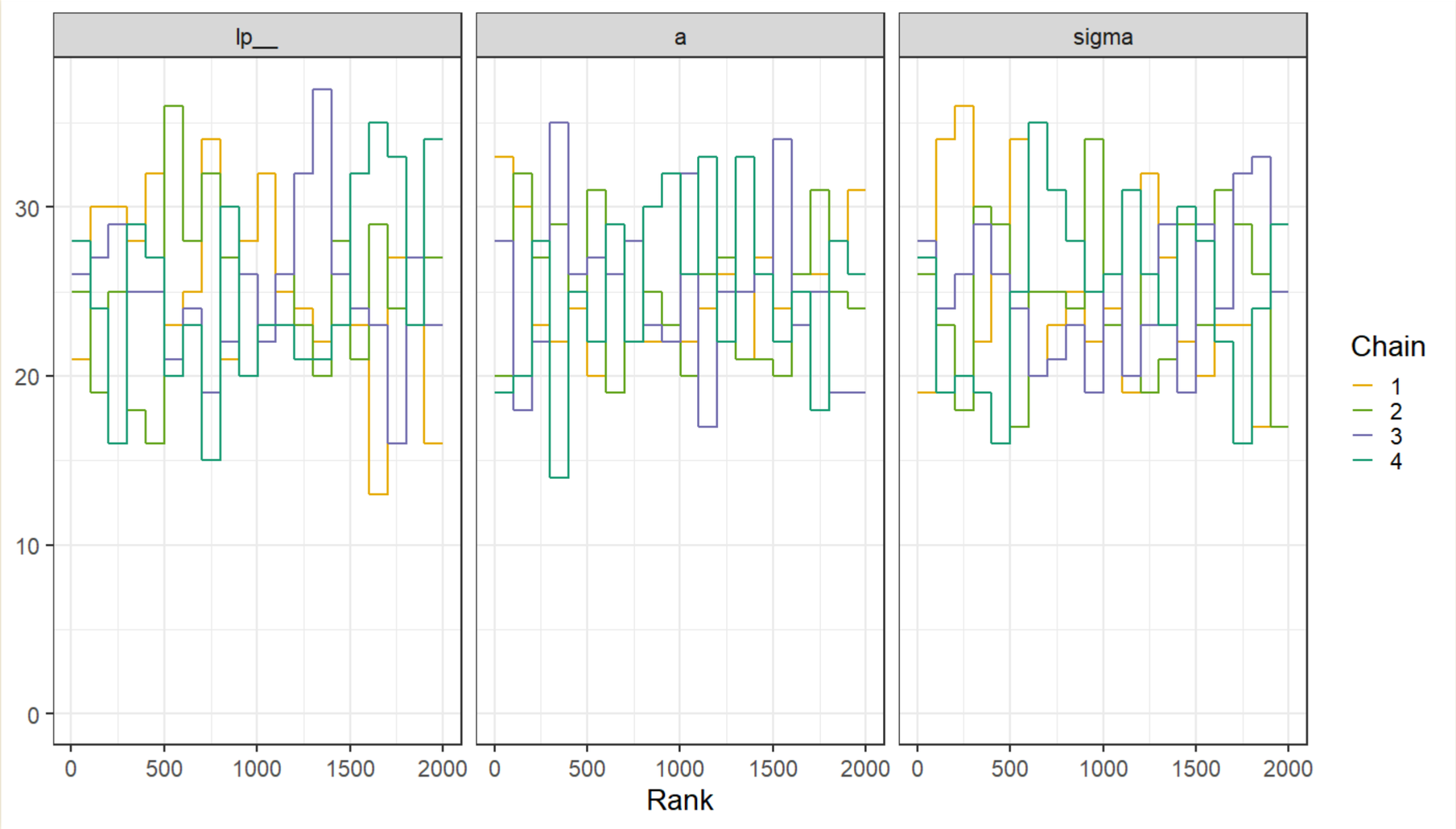
```
color_scheme_set("blue")
mcmc_pairs(re_id_mdl_draws, off_diag_args = list(size = 1, alpha = 0.05))
```



MCMC Trace Plots

```
color_scheme_set("brewer-Dark2")
mcmc_trace(re_id_mdl_draws)
```



MCMC Trace-Rank Plots

```
mcmc_rank_overlay(re_id_mdl_draws)
```